Zero Trust Security with OCI

OUT1015 Practice 3: Inspect the Requests, Not Only the Port

Table of Contents

Zero Trust Security with OCI **1**
Introduction 3
About This Workshop 3
Architecture 3
Get Started 5
Prerequisites 5
Sign In to the Lab Tenancy 5
Sign In and Check Your Region 5
Find Your Participant Number 5
How to Read This Guide 6
Practice 3: Inspect the Requests, Not Only the Port 7
Introduction 7
Objectives 7
Prerequisites 7
Practice 3-1: Meet the Working Shop 7
Practice 3-2: Create a WAF Policy on the Load Balancer 8
Practice 3-3: Add an XSS Rule in Check Mode 9
Practice 3-4: Turn On the WAF Log 11
Practice 3-5: Compare Two Requests in Check Mode 11
Practice 3-6: Block the Pattern 12
Practice 3-7: Limit Repeated Requests 14
Practice 3-8: Read the Evidence 16
Checkpoint: What You Proved 18
Learn More 19
Need Help? 20
Clean Up 21
Remove Your WAF Policy and Log 21
Acknowledgements 22

Copyright © 2026, Oracle and/or its affiliates.

Introduction

About This Workshop

Summit Retail is moving its online store to Oracle Cloud Infrastructure (OCI). Asha leads cloud security for the move. Over one day she handles a series of security incidents, and each one leads to a Zero Trust decision: grant access on purpose, verify every request, and assume that one control can fail.

You watch some incidents as instructor demonstrations, and you fix three of them yourself. Each practice starts with what Asha sees, shows you the part of the architecture involved, and then gives you short tasks. After each task you compare your result with the expected result.

This guide covers **Practice 3: Inspect the Requests, Not Only the Port**. The workshop has four practices, each in its own guide. Do Practice 0 first: it builds the environment that the other practices use. Practices 1, 2, and 3 each need only Practice 0, so you can do them in any order.

Practice	What you do	Time
Practice 0	Prepare your Summit Retail environment (Cloud Shell, Terraform)	20 minutes
Practice 1	Get the password out of the code (OCI Vault, instance principals)	25 minutes
Practice 2	Keep the private application private (Security Lists, NSGs, Bastion)	30 minutes
Practice 3	Inspect the requests, not only the port (Web Application Firewall)	35 minutes

Architecture

Practice 0 builds everything in the diagram except the vault, which the lab team prepared, and the WAF policy, which you add in Practice 3. The badges show where each practice works.

Copyright © 2026, Oracle and/or its affiliates.

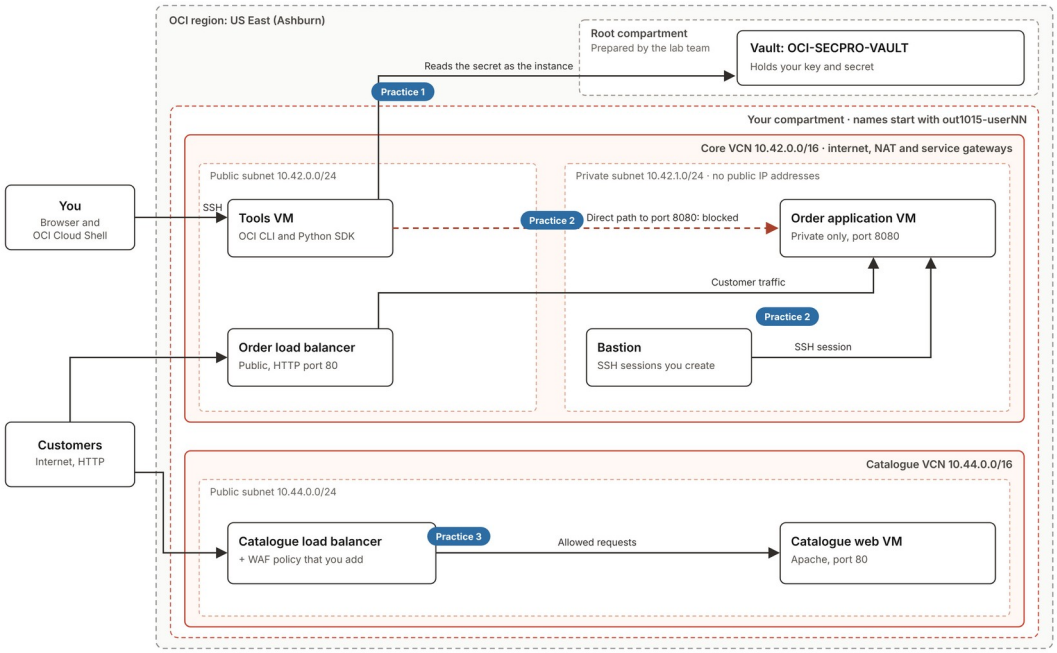


Figure 1: The Summit Retail lab environment in your compartment

The core VCN holds the order system. Customers reach the order application only through the public load balancer. The application server has no public IP address, and you reach it for administration only through OCI Bastion. The Tools VM plays the role of an administrator's or another team's machine inside the same VCN. A second VCN holds the product catalogue, which you protect with WAF.

Copyright © 2026, Oracle and/or its affiliates.

Get Started

Prerequisites

- The lab account details from your instructor: the sign-in URL or tenancy name, your username and password, and your assigned compartment.
- A laptop with a current web browser. You need no other software, because every command runs in OCI Cloud Shell.

Sign In to the Lab Tenancy

Your instructor gives you a lab account in a shared tenancy. Every participant works in their own compartment, so your resources never mix with anyone else's.

Sign In and Check Your Region

1. Open the sign-in URL from your lab details, or go to `https://cloud.oracle.com` and enter the tenancy name.

2. Sign in with the username and password from your lab details. If you are asked to change the password, choose a new one and keep it for the rest of the day.

3. In the Console header, open the region selector and select **US East (Ashburn)**. Every practice uses this region.

4. From the navigation menu, select **Identity & Security**, and then click **Compartments**. Find your assigned compartment in the list, for example `98507690-C07`.

   > **Expected result**
   >
   > You are signed in, the region is US East (Ashburn), and you can see your assigned compartment.

Find Your Participant Number

1. Look at your username. It ends with your participant number, for example `98507690-lab.user07`.

2. Write down the last part, `userNN`, for example `user07`. Everywhere this guide shows `userNN` or `NN`, use your own number.

Copyright © 2026, Oracle and/or its affiliates.

> **Note**
>
> Your compartment uses the same number: user07 works in compartment 98507690-C07. The Practice 0 package reads both from your username, and asks you to confirm them.

How to Read This Guide

You see	It means
A light box with a red left edge	Text that you enter or select in the OCI Console.
A dark box that starts with $	A command to run. The task tells you where: in Cloud Shell or on a VM. Do not type the $.
A gray box	Output that you should see. Details such as IDs, dates, and IP addresses differ for you.
Expected result	What to check before you move on.
`<value>`	A placeholder. Replace it with your own value, without the angle brackets.

> **Note**
>
> Console labels sometimes move between releases. If a button is not where the guide says, look for the same action in the **Actions** menu or under the three-dot icon.

Copyright © 2026, Oracle and/or its affiliates.

Practice 3: Inspect the Requests, Not Only the Port

Introduction

Summit Retail has opened its product catalogue to customers. The network correctly allows HTTP to the catalogue's load balancer. But an ordinary product search and a request carrying an attack pattern arrive on the same allowed port, and the network cannot tell them apart. Then a script starts asking for the catalogue far more often than any customer would.

Asha's fix is OCI Web Application Firewall (WAF) on the load balancer. In this practice you first watch what one WAF rule detects, then let it block the attack pattern, and finally limit how often one client may request the catalogue. The catalogue is a static page that ignores query strings, so the test pattern never runs; it only shows how WAF recognizes and handles it.

Estimated time: 35 minutes.

Objectives

- Create a WAF policy and attach it to the catalogue's load balancer.
- Detect a cross-site scripting (XSS) pattern in Check mode, and confirm the detection in the WAF log.
- Block the pattern with a 403 response while normal requests still work.
- Limit repeated requests with a 429 response, and watch the catalogue recover.

Prerequisites

- Practice 0 is complete, and you have your `catalogue_url`.
- Cloud Shell is open.

Practice 3-1: Meet the Working Shop

1. In Cloud Shell, store your catalogue address and the test pattern, and define a short helper that prints only the HTTP status code. Replace `<catalogue_url>` with your value, which starts with `http://`:

```
$ CAT=<catalogue_url>
$ XSS='q=<script>alert(1)</script>'
$ code() { curl -sS -o /dev/null -w '%{http_code}\n' "$@"; }
```

Copyright © 2026, Oracle and/or its affiliates.

2. Check the catalogue's health:

```
$ curl -sS $CAT/healthz
```

```
summit-retail-web: ok
```

3. In your browser, open two tabs and keep them open for the whole practice:

Tab	Address
Normal customer	`<catalogue_url>/?q=backpack`
Test pattern	`<catalogue_url>/?q=%3Cscript%3Ealert%281%29%3C%2Fscript%3E`

> **Expected result**
>
> Both tabs show the catalogue with three products. Without WAF, the load balancer passes both requests to the web server.

> **Note**
>
> Cloud Shell forgets variables and the helper when it restarts. If a command fails with `code: command not found` or an empty address, run step 1 again.

Practice 3-2: Create a WAF Policy on the Load Balancer

1. From the navigation menu, select **Identity & Security** > **Web Application Firewall**, and then click **Policies**.

2. Next to **Applied filters**, select your assigned compartment. Click **Create WAF policy**.

3. Enter these values, and click **Create**:

Field	Value
Name	`out1015-userNN-waf-policy`
Compartment	Your assigned compartment

4. Wait until the policy is **Active**. On the policy details page, click the **Firewalls** tab. Click **Actions**, and then **Add Firewalls**.

5. Enter these values, and click **Add Firewalls**:

Field	Value
Firewall name	`out1015-userNN-firewall`

Copyright © 2026, Oracle and/or its affiliates.

Field	Value
Create in compartment	Your assigned compartment
Load balancer compartment	Your assigned compartment
Load balancer	`out1015-userNN-waf-lb`

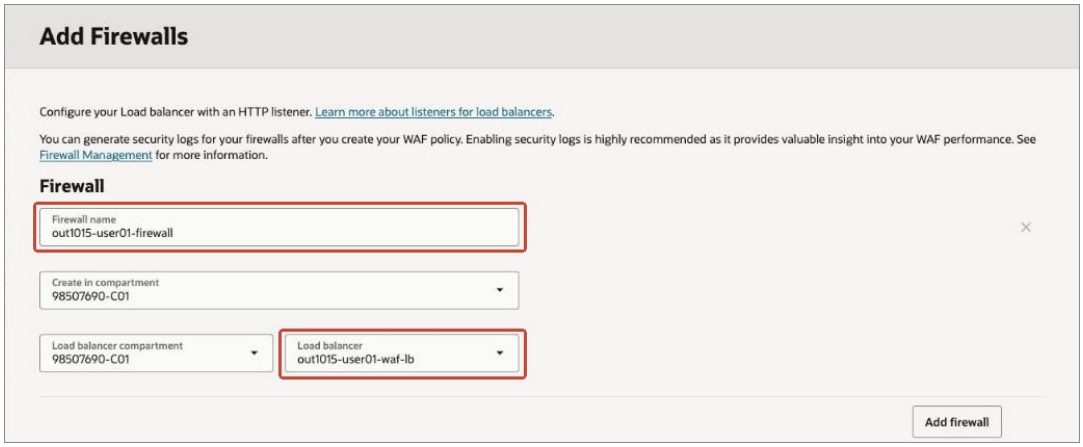


Figure 2: Add Firewalls: attach the policy to the catalogue load balancer

6. Wait until the firewall's **Attachment state** is **Active**. It takes a minute or two.

> **Note**
>
> Select the load balancer whose name ends in `waf-lb`. The order load balancer from Practice 2 stays without WAF.

Practice 3-3: Add an XSS Rule in Check Mode

Check mode records a match but lets the request through. It shows you what a rule would catch before you let it block anything.

1. On the policy details page, click the **Protections** tab. Click **Actions**, and then **Manage request protection rules**. Click **Add request protection rule**.

2. Enter these values:

Field	Value
Name	`inspect-xss`
Conditions	Do not add any condition; leave the empty **Path** row as it is
Rule action	**Pre-configured Check Action**
Body inspection	Leave disabled

Copyright © 2026, Oracle and/or its affiliates.

3. Under **Protection capabilities**, click **Actions**, and then **Choose protection capabilities**. Next to **Applied filters**, remove the **Tags: Recommended** filter by clicking its **x**; otherwise the search finds nothing. Search for `941110`, select **941110: Cross-Site Scripting (XSS) Attempt: XSS Filters - Category 1**, and click **Choose protection capabilities**.

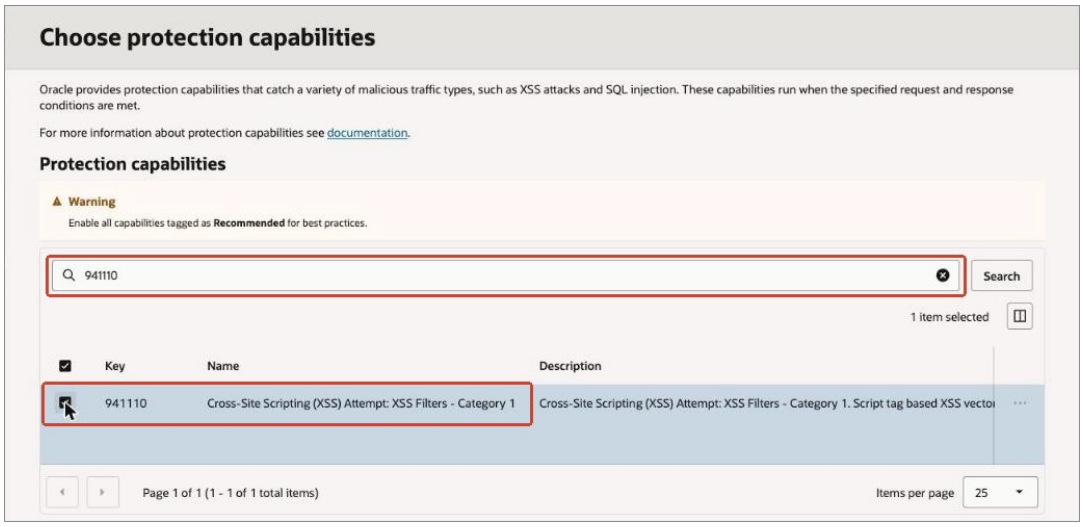


Figure 3: Choose protection capability 941110

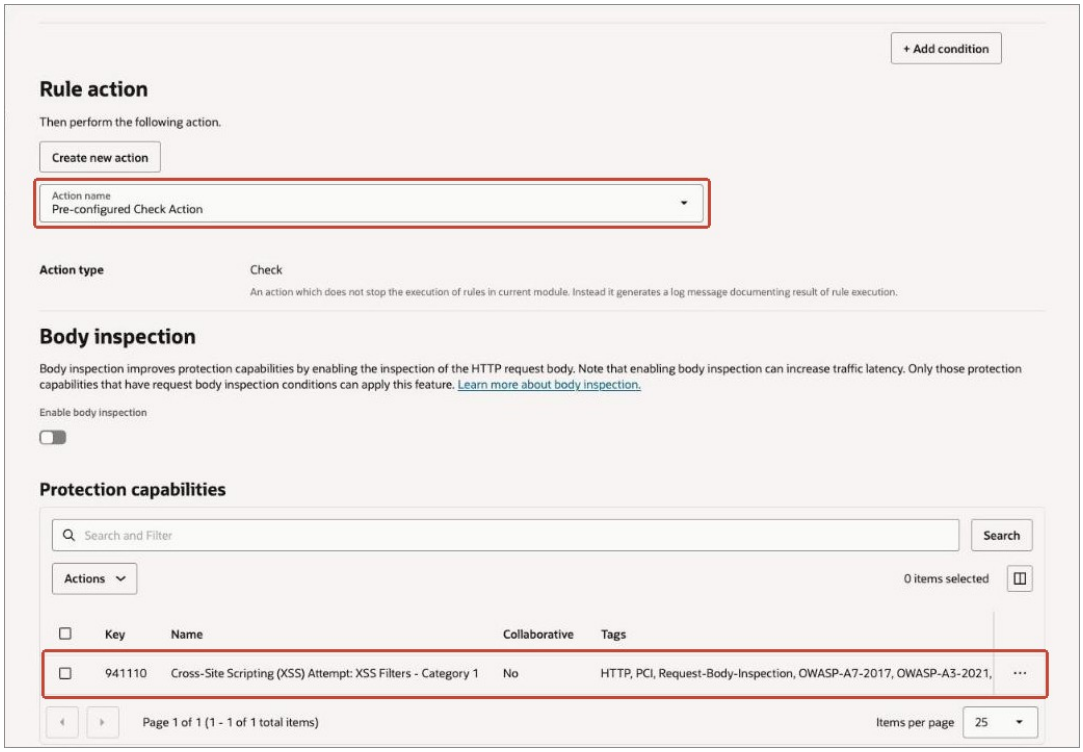


Figure 4: The rule uses the Pre-configured Check Action and capability 941110

Copyright © 2026, Oracle and/or its affiliates.

4. Click **Add request protection rule**, and then click **Save changes**. Wait until the policy is **Active** again.

Practice 3-4: Turn On the WAF Log

1. On the policy details page, click the **Firewalls** tab, and click `out1015-userNN-firewall`.
2. Click the **Monitoring** tab. In the **Logs** section, click the three dots in the **All Logs** row, and click **Enable log**.
3. Enter these values, and click **Enable log**:

Field	Value
Compartment	Your assigned compartment
Log group	Create a new group named `out1015-userNN-logs`
Log name	Keep the default, `out1015-userNN-firewall_all`
Log retention	**1 month**

4. Wait until the log is **Active**.

Practice 3-5: Compare Two Requests in Check Mode

1. Reload both browser tabs. Both still show the catalogue.
2. In Cloud Shell, check the exact status codes:

```
$ code "$CAT/?q=backpack"
$ code -G --data-urlencode "$XSS" $CAT/
```

```
200
200
```

3. Open the WAF log: on the firewall's **Monitoring** tab, click `out1015-userNN-firewall_all` in the **Logs** section. The log opens in a new browser tab; click its **Explore log** tab. Log entries can take a few minutes to appear.
4. In the **Search and Filter** box, enter `inspect-xss`, and click **Search**. If nothing is listed, click **Edit** next to **Time range**, select **Past 30 minutes**, and click **Update**.
5. Expand the newest entry, and look at these fields:

Field	What you see
`action`	`allow`

Copyright © 2026, Oracle and/or its affiliates.

Field	What you see
`requestProtection > matchedRules`	`inspect-xss`
`requestProtection > matchedIds`	`941110_v002`
`response > code`	`200`

> **Expected result**
>
> WAF detected the pattern and let the request through: Check mode records the match and still allows the request. The normal request has no `requestProtection` match.

Practice 3-6: Block the Pattern

1. On the policy details page, click the **Protections** tab. Click **Actions**, and then **Manage request protection rules**. Click the three dots next to `inspect-xss`, and click **Edit**.

2. Under **Rule action**, click **Create new action**, and enter these values:

Field	Value
Name	`block-xss`
Type	**Return HTTP response**
Response code	**403 Forbidden**
Response header	`Content-Type` with value `text/plain; charset=utf-8`
Response page body	`Summit Retail WAF: request blocked by XSS rule`

Copyright © 2026, Oracle and/or its affiliates.

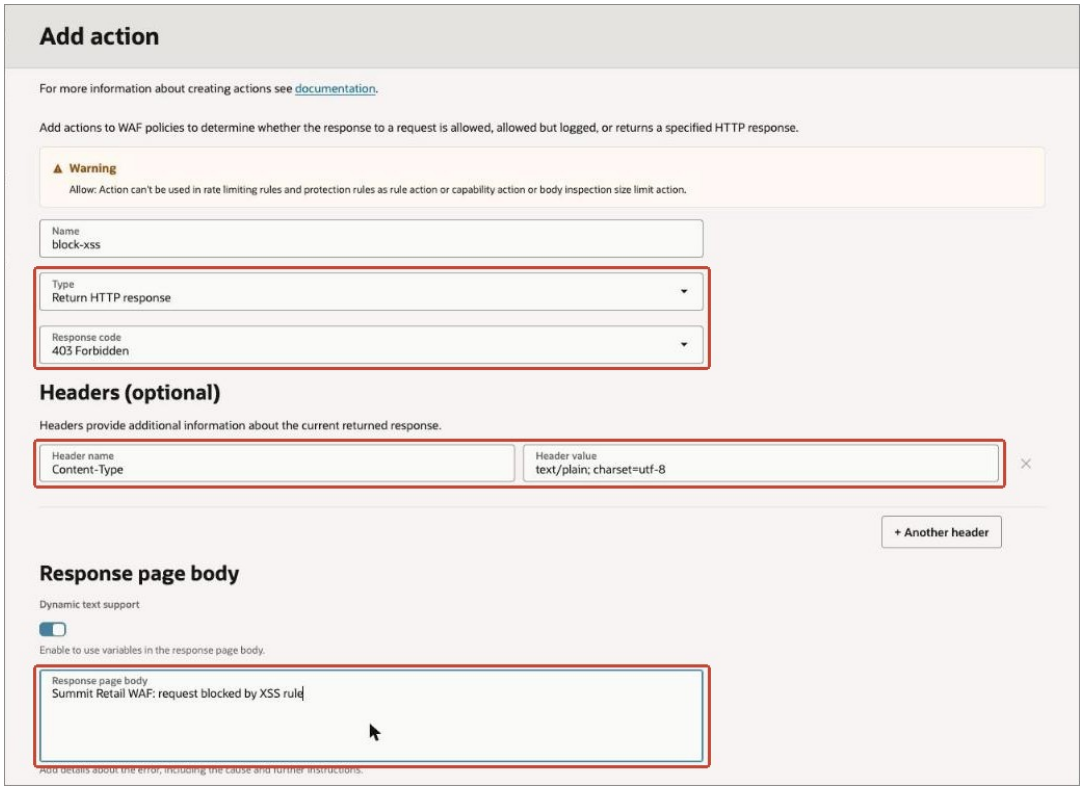


Figure 5: The block-xss action returns 403 with a plain-text message

3. Click **Add action**. The new action is now selected under **Action name**. Keep capability 941110 as it is, click **Update**, and then click **Save changes**. Wait until the policy is **Active** again.

4. Test both requests and the health check:

```
$ code "$CAT/?q=backpack"
$ code -G --data-urlencode "$XSS" $CAT/
$ curl -sS -G --data-urlencode "$XSS" $CAT/; echo
$ curl -sS $CAT/healthz
```

```
200
403
Summit Retail WAF: request blocked by XSS rule
summit-retail-web: ok
```

5. Reload the test-pattern tab in your browser with a hard refresh (**Ctrl+Shift+R**, or **Cmd+Shift+R** on a Mac). It now shows the block message, while the normal tab still shows the catalogue.

Copyright © 2026, Oracle and/or its affiliates.

> **Note**
>
> If the pattern still returns `200`, the policy change is still being deployed. Wait a minute and test again.

Practice 3-7: Limit Repeated Requests

The next request looks ordinary. The problem is how often it comes: a script is asking for the catalogue many times a second.

1. On the policy details page, click the **Rate limiting** tab. Click **Actions**, and then **Manage rate limiting**. Click **Add rate limiting rule**.

2. Enter these values:

Field	Value
Name	`limit-catalogue-requests`
Conditions	Condition type **Path**, operator **Is**, value `/`
Requests limit	`5`
Period in seconds	`10`
Action duration in seconds	`30`

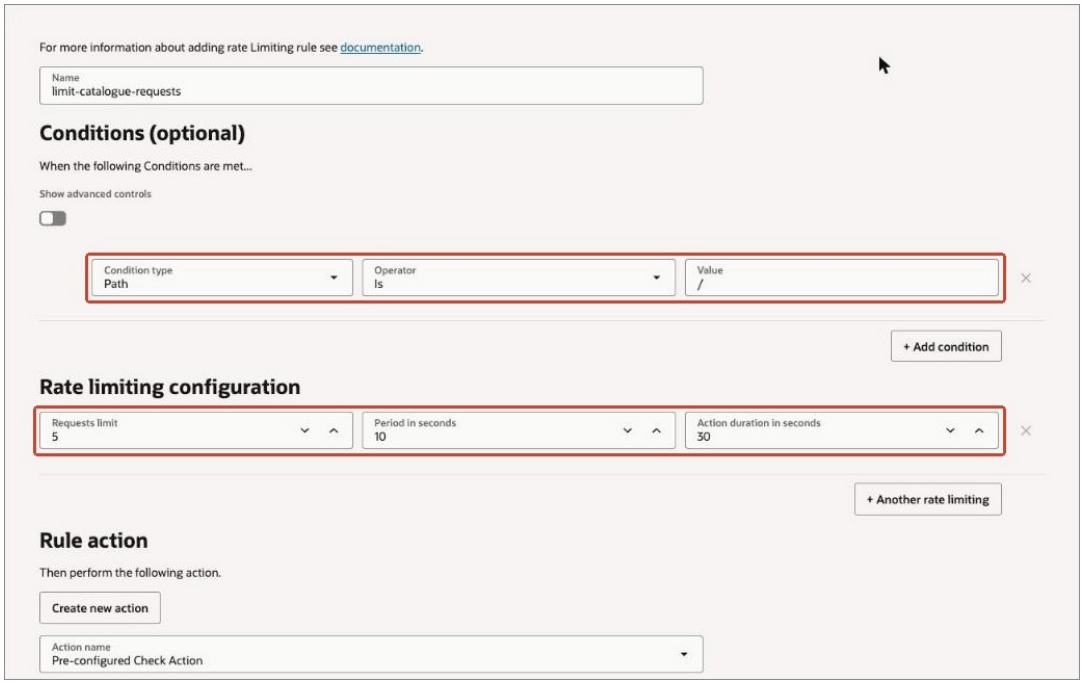


Figure 6: The rate limiting rule: path /, 5 requests in 10 seconds, then 30 seconds of 429 responses

Copyright © 2026, Oracle and/or its affiliates.

3. Under **Rule action**, click **Create new action**, and enter these values. Then click **Add action**:

Field	Value
Name	`too-many-requests`
Type	**Return HTTP response**
Response code	**429 Too Many Requests**
Response header	`Content-Type` with value `text/plain; charset=utf-8`
Response page body	`Too many catalogue requests. Please try again shortly.`

4. Click **Add rate limiting rule**, and then click **Save changes**. Wait until the policy is **Active** again.

5. From Cloud Shell, send ten requests in quick succession, then check the health path:

```
$ for i in $(seq 10); do code "$CAT/?q=backpack"; done
$ curl -sS $CAT/healthz
```

```
200
200
200
200
429
200
429
...
summit-retail-web: ok
```

6. Stop sending requests for 40 seconds, then check that the catalogue is back, and that the XSS rule still blocks the pattern:

```
$ code "$CAT/?q=backpack"
$ code -G --data-urlencode "$XSS" $CAT/
```

```
200
403
```

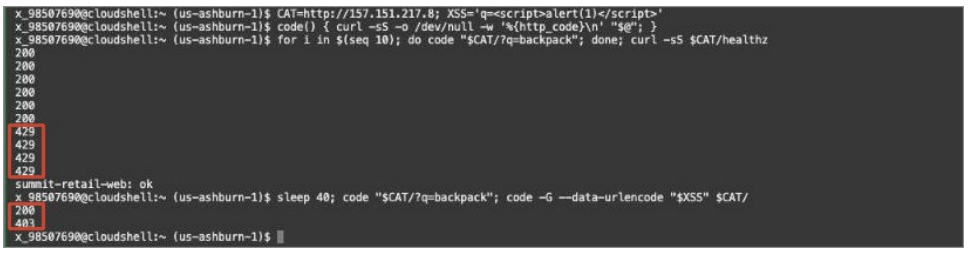


Figure 7: In Cloud Shell: 429 after a burst, then 200 again after a pause, while the XSS pattern stays at 403

Copyright © 2026, Oracle and/or its affiliates.

> **Expected result**
>
> After a few ordinary responses, `429` responses appear. The exact sequence varies, because the limit is counted per client and is not exact to the request. `/healthz` stays at `200`, because the rule applies only to the path `/`.

> **Note**
>
> Run the loop from Cloud Shell. Some networks send your requests out through several addresses, which spreads the count and can hide the limit.

Practice 3-8: Read the Evidence

1. Open the WAF log again, as in Practice 3-5. Clear the search box, click **Search**, and expand the newest entries.

   > **Note**
   >
   > Your load balancer is on the internet, so the log can also hold requests from addresses that are not yours, such as scanners probing common paths. Your own tests come from the Cloud Shell address shown in `clientAddr`.

2. Match each test to its log entry:

Test	`action`	Response code	Rule
Normal request	`allow`	200	None
XSS pattern	`block`	403	`inspect-xss` (`requestProtection`)
Too many requests	`block`	429	`limit-catalogue-requests` (`requestRateLimiting`)

Copyright © 2026, Oracle and/or its affiliates.

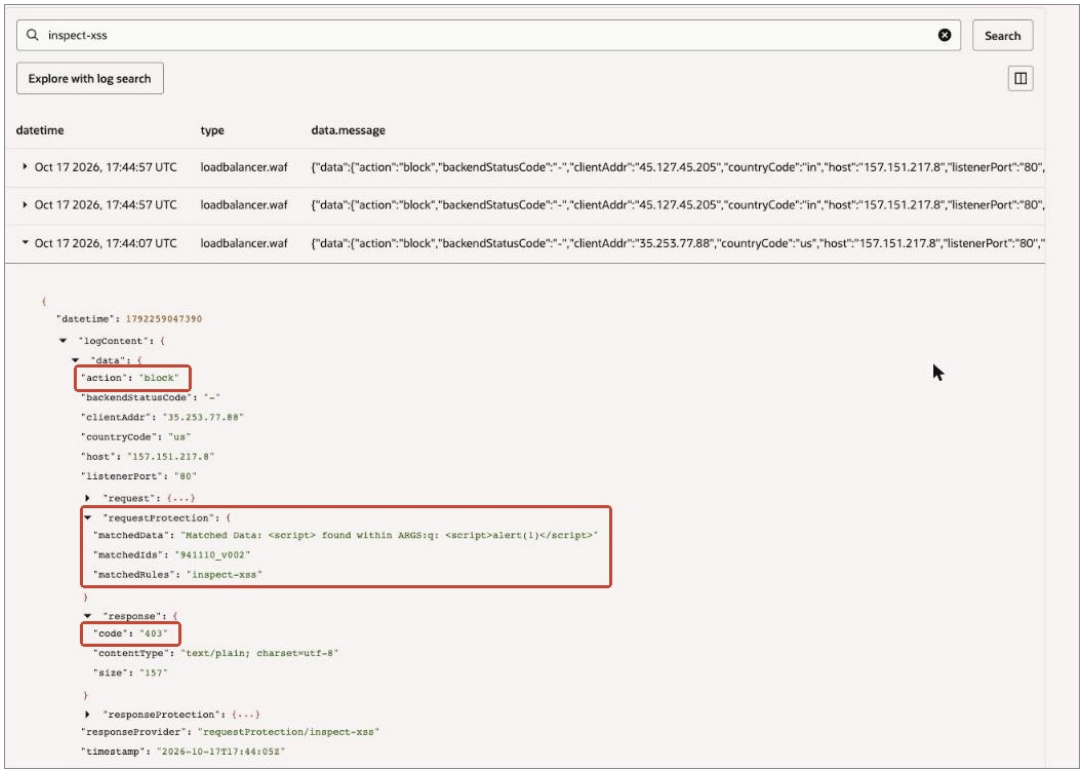


Figure 8: A blocked request in the WAF log: action block, the matched rule, and response 403

Expected result

Each response that you saw in Cloud Shell has a log entry that names the rule and the action. That record is how Asha shows what WAF did, and why.

3. (Optional) Go back to the firewall's **Monitoring** tab. The **Blocked requests** chart counts the 403 and 429 responses over time.

Copyright © 2026, Oracle and/or its affiliates.

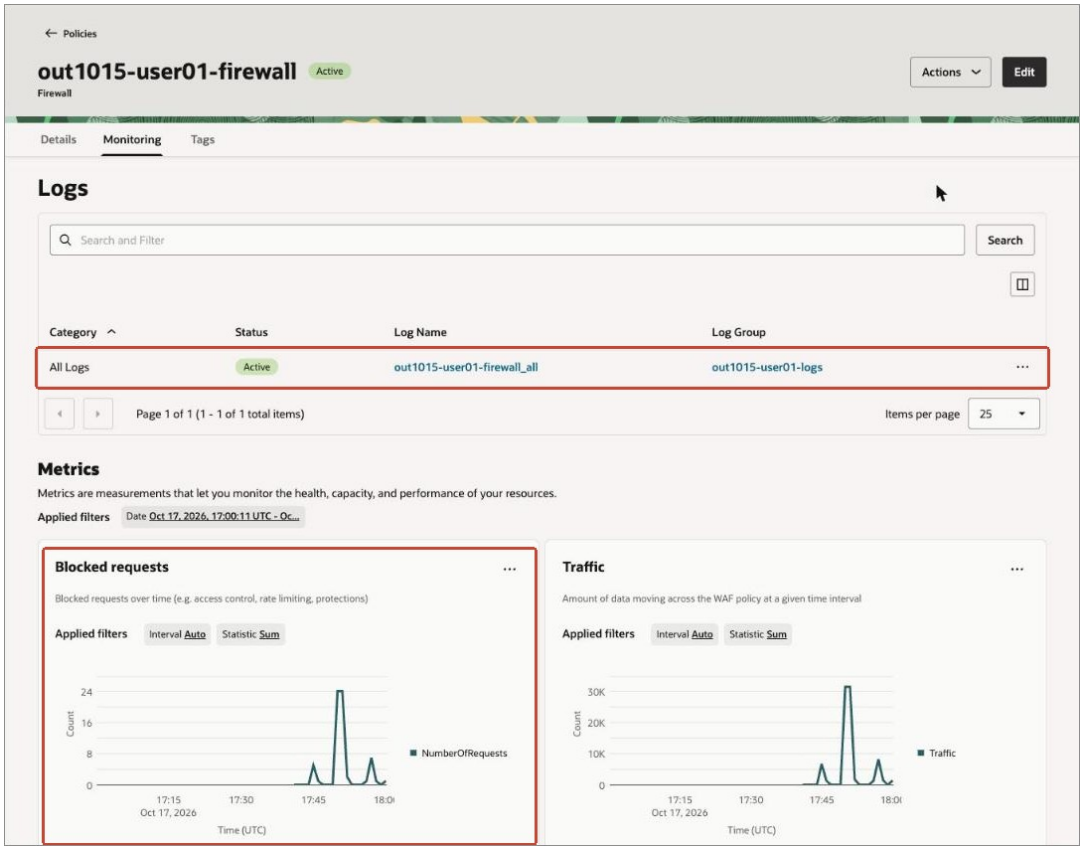


Figure 9: The firewall's Monitoring tab: the WAF log and the Blocked requests metric

Checkpoint: What You Proved

Request	No WAF	Check mode	Blocking	Rate limit active
`?q=backpack`	200	200	200	200, then 429
XSS pattern	200	200 (logged)	403	403
`/healthz`	200	200	200	200

- Why start in Check mode instead of blocking straight away?
- The XSS rule blocks this pattern. Does that make the application safe from XSS?
- Why does `/healthz` keep working while the catalogue returns 429?

Copyright © 2026, Oracle and/or its affiliates.

> **Checkpoint**
>
> **1.** Check mode shows what a rule matches on real traffic, so you find false positives before customers are blocked.
>
> **2.** No. One rule catches known patterns. The application still needs input handling, patching, and its own security testing.
>
> **3.** The rate rule's condition matches only the path `/`. Health checks and other paths are not counted.

Learn More

- OCI Web Application Firewall overview: https://docs.oracle.com/en-us/iaas/Content/WAF/Concepts/overview.htm
- Rate limiting rules: https://docs.oracle.com/en-us/iaas/Content/WAF/RateLimiting/create_rate_limiting_rule.htm

Copyright © 2026, Oracle and/or its affiliates.

Need Help?

Raise your hand, and an instructor will come to you. Before that, check the common problems below.

- `code: command not found`: Cloud Shell restarted. Run Practice 3-1, step 1 again.
- `502 Bad Gateway`: the web server is not ready or not healthy. Check `curl -sS $CAT/healthz`, wait a minute, and tell your instructor if it does not recover. A 502 is not a WAF block.
- The XSS pattern still returns `200` after Practice 3-6: check that `inspect-xss` uses the action `block-xss`, that the policy is Active, and that the firewall is attached to `out1015-userNN-waf-lb`. Then send a fresh request.
- No `429` in Practice 3-7: check that the rule's condition is the path `/`, wait until the policy is Active, and run the loop again from Cloud Shell.
- The log is empty: check that the log is enabled on the firewall, and wait a few minutes. Log entries arrive after the HTTP response.

Copyright © 2026, Oracle and/or its affiliates.

Clean Up

Clean up when your instructor tells you to, normally at the end of the day. The lab resources incur charges while they exist.

Remove Your WAF Policy and Log

1. From the navigation menu, select **Identity & Security** > **Web Application Firewall**, and then click **Policies**. Next to **Applied filters**, select your assigned compartment.

2. Click `out1015-userNN-waf-policy`. Click the **Firewalls** tab, select `out1015-userNN-firewall`, and click **Actions** > **Delete**. Confirm, and wait until the firewall is deleted.

3. At the top of the policy page, click **Actions** > **Delete**, and confirm.

4. From the navigation menu, select **Observability & Management** > **Logging**, and then click **Log Groups**. Click `out1015-userNN-logs`, delete the log `out1015-userNN-firewall_all`, and then delete the log group.

Copyright © 2026, Oracle and/or its affiliates.

Acknowledgements

- **Author:** Saurabh Patil, Principal OCI Instructor, Oracle University
- **Workshop:** OUT1015 Zero Trust Security with OCI, Oracle AI World 2026
- **Last updated:** October 2026

Copyright © 2026, Oracle and/or its affiliates.